

学号：202300130183	姓名：宋浩宇
实验名称：实验二十一-文本摘要生成实验——基于预训练语言模型的自动摘要	
1. 实验目的	
- 理解自动文本摘要（Text Summarization）的基本概念。 - 掌握摘要生成任务的基本形式。 - 学习使用预训练语言模型进行摘要生成。 - 理解摘要与原文之间的语义关系。 - 分析自动摘要生成中的信息保留与语义压缩问题。	
2. 实验原理	
自动摘要生成的目标是： 在保留核心语义信息的前提下，自动生成较短文本。 例如： 原文： 人工智能正在快速发展，并在医疗、教育、金融等领域发挥重要作用。 摘要： 人工智能正在多个领域快速发展。 文本摘要通常分为两类： - 抽取式摘要（Extractive Summarization） 从原文中抽取关键句子作为摘要。 - 生成式摘要（Abstractive Summarization） 模型理解原文后重新生成摘要。 现代摘要系统通常基于 Transformer 或大语言模型实现，例如： • BART • T5 • GPT 这些模型能够学习文本中的重要信息，并生成更加自然流畅的摘要。	
3. 实验步骤与代码	
实验步骤： - 导入 transformers 与 torch 库 - 加载预训练摘要模型（如 T5、BART） - 输入原始文本 - 对文本进行编码 - 调用模型生成摘要 - 对输出结果进行解码 - 输出摘要结果 - 修改输入文本重新测试 - 分析摘要质量与长度变化	
代码： <pre># -*- coding: utf-8 -*- from __future__ import annotations</pre>	

```

import os
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parent))
import week10_runtime
week10_runtime.setup_runtime_env()
BASE_DIR = Path(__file__).resolve().parent
DATA_PATH = BASE_DIR / "exp-10.2-data.txt"
SUM_MODEL = os.environ.get("NLP_WEEK10_SUM_MODEL",
"fnlp/bart-base-chinese")
MAX_LEN_LONG = int(os.environ.get("NLP_WEEK10_SUM_MAX_LEN", "64"))
MAX_LEN_SHORT = int(os.environ.get("NLP_WEEK10_SUM_MAX_LEN_SHORT",
"24"))
def parse_data(path: Path) -> dict[str, str]:
    blocks: dict[str, str] = {}
    mode = None
    buf: list[str] = []
    for raw in path.read_text(encoding="utf-8-sig").splitlines():
        line = raw.strip()
        if line.startswith("[") and line.endswith("]"):
            if mode and buf:
                blocks[mode] = "".join(buf)
                mode = line[1:-1]
                buf = []
            continue
        if mode and line:
            buf.append(line)
    if mode and buf:
        blocks[mode] = "".join(buf)
    return blocks
def build_sum_model():
    import torch
    from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
    token = week10_runtime.hub_token()
    kw = {"trust_remote_code": True}
    if token:
        kw["token"] = token
    local_path = week10_runtime.download_model(SUM_MODEL)
    print(f"模型目录: {local_path}")
    device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
    tok = AutoTokenizer.from_pretrained(local_path, **kw)
    model = AutoModelForSeq2SeqLM.from_pretrained(local_path,
**kw)

```

```

        model.to(device)
        model.eval()
        return tok, model, device
def gen_summary(tok, model, device, text: str, max_length: int,
min_length: int = 8) -> str:
    import torch
    inputs = tok(text[:1024], return_tensors="pt", truncation=True,
max_length=512)
    gen_in = {k: v.to(device) for k, v in inputs.items() if k in
("input_ids", "attention_mask")}
    with torch.inference_mode():
        out_ids = model.generate(
            **gen_in,
            max_length=max_length,
            min_length=min_length,
            num_beams=4,
            early_stopping=True,
        )
    return tok.decode(out_ids[0],
skip_special_tokens=True).strip().replace(" ", "")
def summarize(tok, model, device, title: str, text: str, max_length:
int, min_length: int = 8) -> None:
    if not text:
        return
    print(f"\n--- {title} ---")
    print(f"原文({len(text)}字): {text}")
    summary = gen_summary(tok, model, device, text, max_length,
min_length)
    print(f"摘要({len(summary)}字, max_length={max_length}):
{summary}")
def main() -> None:
    print(f"数据文件: {DATA_PATH}")
    print(f"模型: {SUM_MODEL}")
    week10_runtime.print_runtime_info()
    if not DATA_PATH.is_file():
        print("数据文件不存在。")
        raise SystemExit(1)
    blocks = parse_data(DATA_PATH)
    try:
        tok, model, device = build_sum_model()
    except Exception as e:
        print(f"模型加载失败: {type(e).__name__}: {e}")
        print("提示: 请确认代理已开启;所有下载与临时文件应位于
HF_HOME 与 TMP 所示目录。")

```

```

        print("      C 盘旧目
录 %USERPROFILE%\\.cache\huggingface 可手动删除释放空间。")
        print("      也可双击运行：配置代理并运行第十周实验.bat
21")

        raise SystemExit(1) from e
    t1 = blocks.get("文本一", "")
    t2 = blocks.get("文本二", "")
    t_short = blocks.get("短文本", "")
    if t1:
        summarize(tok, model, device, "文本一", t1, MAX_LEN_LONG)
    if t2:
        summarize(tok, model, device, "文本二", t2, MAX_LEN_LONG)
    if t_short:
        summarize(tok, model, device, "短文本", t_short,
MAX_LEN_LONG)
        summarize(tok, model, device, "短文本(较短 max_length)",
t_short, MAX_LEN_SHORT)
if __name__ == "__main__":
    main()

```

4. 实验结果（图表与输出）

输出结果为：

```

F:\Homework\自然语言处理作业\venv\Scripts\python.exe F:\Homework\自然语言处理作业
\第十周实验\实验二十一-文本摘要生成实验——基于预训练语言.py
数据文件: F:\Homework\自然语言处理作业\第十周实验\exp-10.2-data.txt
模型: fnlp/bart-base-chinese
代理: http://127.0.0.1:7897
HF_HOME: F:\Downloads\huggingface
HF_HUB_CACHE: F:\Downloads\huggingface\hub
TMP: F:\Downloads\huggingface\_tmp
HF_ENDPOINT: https://huggingface.co
F:\Homework\自然语言处理作业\venv\lib\site-packages\torch\cuda\__init__.py:61:
FutureWarning: The pynvml package is deprecated. Please install nvidia-ml-py instead. If you
did not install pynvml directly, please report this to the maintainers of the package that installed
pynvml for you.
    import pynvml # type: ignore[import]
第 1 轮 Endpoint=https://hf-mirror.com 代理=关
模          型          目          录          :
F:\Downloads\huggingface\hub\models--fnlp--bart-base-chinese\snapshots\ffbee86596e8b14cfb
1ef20d4cafa2ea30d272f8
Loading weights: 100%[████████████████████] 260/260 [00:00<00:00, 9771.59it/s]

--- 文本一 ---
原文(80 字): 人工智能正在快速发展，并逐渐应用于医疗、教育、金融和交通等多个领域。

```

随着深度学习技术的进步，大语言模型在文本生成、机器翻译和智能问答等任务中表现出了强大的能力。 摘要(121字,max_length=64): 人工智能正在快速发展，并逐渐应用于医疗、教育、金融和交通等多个领域中随着深度学习技术的进步，大语言模型在文本生成、机器翻译 --- 文本二 --- 原文(68字): 机器学习是人工智能的重要分支，其核心思想是通过数据自动学习规律。近年来，深度学习在图像识别、语音识别和自然语言处理等领域取得了显著成果。 摘要(121字,max_length=64): 机器学习是人工智能的重要分支，其核心思想是通过数据自动学习规律研究近年来，深度学习在图像识别、语音识别和自然语言处理等领域 --- 短文本 --- 原文(19字): 深度学习在图像识别领域取得了显著成果。 摘要(37字,max_length=64): 深度学习在图像识别领域取得了显著成果。 --- 短文本（较短 max_length） --- 原文(19字): 深度学习在图像识别领域取得了显著成果。 摘要(37字,max_length=24): 深度学习在图像识别领域取得了显著成果。 进程已结束，退出代码为 0

5. 结果分析
1. 摘要是否保留了原文核心信息？ 答: 核心信息大体保留，但压缩效果有限。文本一原文 80 字，摘要约 121 字(max_length=64)，内容仍围绕人工智能快速发展、应用于医疗教育金融交通、深度学习进步、大语言模型在文本生成机器翻译智能问答等能力展开，与原文主题一致，句末在"机器翻译"处截断。文本二原文 68 字，摘要约 121 字，保留机器学习是人工智能分支、通过数据学习规律、深度学习在图像语音与自然语言处理等领域取得成果等要点，同样在句末未完整收束。短文本原文 19 字，两种 max_length 下摘要均为对原句的近乎完整复述。整体可认为核心语义未偏离，但摘要偏长，压缩不明显。 2. 哪些信息被保留？哪些信息被省略？ 答: 文本一保留了"人工智能快速发展""多领域应用""深度学习技术进步""大语言模型及文本生成、机器翻译、智能问答"等要素；因截断，句末"智能问答等任务中表现出了强大的能力"后半部分未完整出现。文本二保留了"机器学习与人工智能关系""数据自动学习规律""深度学习在图像识别、语音识别、自然语言处理等领域的成果"；"近年来"等措辞在输出中有"研究近年来"类改写，细节略有变化。短文本几乎全部保留"深度学习在图像识别领域取得了显著成果"。省略主要体现在未对长句做大幅概括，而是以复述与截断代替明显删减，未单独

突出一句式极简摘要。

3. 摘要长度是否会影响信息完整性？

答：对本次短文本几乎无影响。短文本在 `max_length=64` 与 `max_length=24` 两种设置下，摘要均为 37 字且内容相同，说明当原文很短、已低于生成长度下限时，降低 `max_length` 不会改变输出。对文本一、文本二，`max_length=64` 时摘要已达约 121 字且仍被截断，若进一步缩短 `max_length`，预计会丢失更多句末信息。故长度参数在长文本上更可能影响完整性，在短文本上本次未体现出差异。

4. 生成式摘要与抽取式摘要有哪些区别？

答：本实验使用 `fnlp/bart-base-chinese`，属于生成式摘要：模型基于编码器解码器结构生成新 `token` 序列，输出不必与原文连续子串完全一致。当次结果虽大量复述原文用词，但仍可能出现轻微改写，例如文本二摘要中的“研究近年来”与原文“近年来”不完全相同。抽取式摘要则直接从原文选句或选 `span` 拼接，保证每字来自原文。生成式更灵活，也可能像本次这样在训练数据或解码参数影响下偏向冗长复述。

5. 模型是否会生成不准确或不存在的信息？

答：本次未见明显“幻觉”式虚构事实，主要内容均可在原文中找到对应。文本二摘要将“近年来”处理为“研究近年来”，属于轻微表述变化，不算完全无依据，但也不是严格照抄。文本一、二摘要句末不完整，属于生成长度与截断导致的形式问题，而非凭空捏造新事实。短文本与原文几乎一致，未出现原文没有的领域或数字。

6. 长文本摘要是否比短文本更困难？为什么？

答：从本次输出看，长文本摘要更难得到简洁、完整的句子。文本一、二原文分别为 80 字与 68 字，摘要反而更长且句末被截断，说明模型在有限 `max_length` 下难以在一句内收束全部要点。短文本 19 字则几乎被全文复制，任务较简单。原因包括：长文本含多个并列信息点，需要筛选与压缩；解码 `max_length`、`min_length` 与 `beamsearch` 参数会限制输出形态；本模型在中文短段落上可能更倾向保留多数原词。故长文本对摘要系统的信息选择与长度控制要求更高。

7. 为什么 Transformer 模型适合摘要生成任务？

答：BART 基于 Transformer 编码器解码器，编码阶段可对全文建立长距离依赖表示，解码阶段可逐 `token` 生成摘要，适合将长输入映射为短输出。自注意力有助于同时关联“人工智能”“多领域应用”“大语言模型”等分散信息。相比纯 RNN，并行计算与预训练权重有利于在有限样本上获得可读的中文摘要。本次实验能加载 `fnlp/bart-base-chinese` 并对三段文本完成生成，说明该架构可端到端完成摘要推理；效果上仍受 `max_length` 与模型微调数据影响，需结合参数调节评价质量。

6. 实验总结

本次在加载 `fnlp/bart-base-chinese`，对任务书中两段长文本与一段短文本完成摘要生成，程序正常结束，退出代码 0。长文本摘要保留主题但压缩不足且句末截断；短文本在 `max_length=64` 与 24 下输出相同。

使用 LLM 辅助完成代码编写。